

Prueba Gemini API

Una API es como un puente que permite que dos sistemas distintos se comuniquen entre sí sin necesidad de que uno entienda todos los detalles internos del otro. En la práctica, sirve para que aplicaciones compartan información o funciones de manera ordenada. Por ejemplo, cuando una app de viajes te muestra vuelos, en realidad está consultando la API de las aerolíneas para traer los datos en tiempo real.

El prompt engineering es básicamente el arte de saber cómo pedirle cosas (diseñar, estructurar y optimizar las instrucciones de texto) a un modelo de IA para que te dé la salida que necesitas. Importa porque la calidad de la respuesta depende mucho de cómo se formule la instrucción. Una práctica sencilla es dar contexto claro y ejemplos en el mismo prompt, lo que ayuda a que el modelo entienda mejor la intención, también se puede utilizar una herramienta de IA de respuesta rápida para crear una instrucción más precisa que después será el prompt para el modelo. En procesos de negocio, esto significa que puedes obtener resultados más consistentes y útiles, reduciendo errores y ahorrando tiempo.

Usar una herramienta de IA de forma manual es más interactivo y puntual: escribes una consulta y recibes una respuesta, pero dependes de hacerlo uno por uno. En cambio, integrarla vía API en un script permite automatizar tareas, escalar a miles de consultas y tener control sobre cómo se procesan los resultados. Esto abre la puerta a flujos de trabajo más robustos, donde la IA se convierte en parte de un sistema completo en lugar de ser solo una herramienta aislada. La diferencia clave está en la eficiencia y en la capacidad de integración con otros procesos.

La prueba trata de analizar un archivo "respuestas_encuesta.csv" con 20 respuestas abiertas a la pregunta: "¿Qué tan satisfecho estás con el servicio de atención al cliente?"
Escribir un script en Python que:

- Lea el archivo CSV
- Por cada respuesta, llame a la API de Gemini y le pida clasificar el sentimiento como Positivo, Negativo o Neutro.
- Guarde los resultados en un nuevo archivo "resultados_sentimiento.csv" con las columnas: id, respuesta, sentimiento.
- Imprima en consola un resumen: cuántas respuestas fueron Positivas, Negativas y Neutras.

Cómo estructuré mi prompt:

Lo diseñé para que fuera claro y restrictivo: pedí que la salida fuera únicamente una palabra entre "Positivo", "Negativo" o "Neutro"
Esto ayudó a mantener consistencia y evitar respuestas largas o ambiguas.

Dificultades encontradas:

Durante la ejecución tuve problemas con la disponibilidad de modelos: varios aparecían listados pero devolvían errores 404, por lo que tuve que probar alternativas hasta llegar a 'gemini-3-flash-preview'.

También me encontré con límites de cuota en el free tier (error 429), lo que interrumpió algunas llamadas y consumió mis tokens más rápido de lo esperado.

Qué mejoraría:

Implementaría un sistema de control de tasa más inteligente para evitar exceder los límites de la API gratuita, quizá con colas o batch processing.

También añadiría un caché local para no repetir llamadas sobre textos ya procesados y optimizar el uso de tokens. Explorar modelos más recientes y estables para asegurar mayor confiabilidad